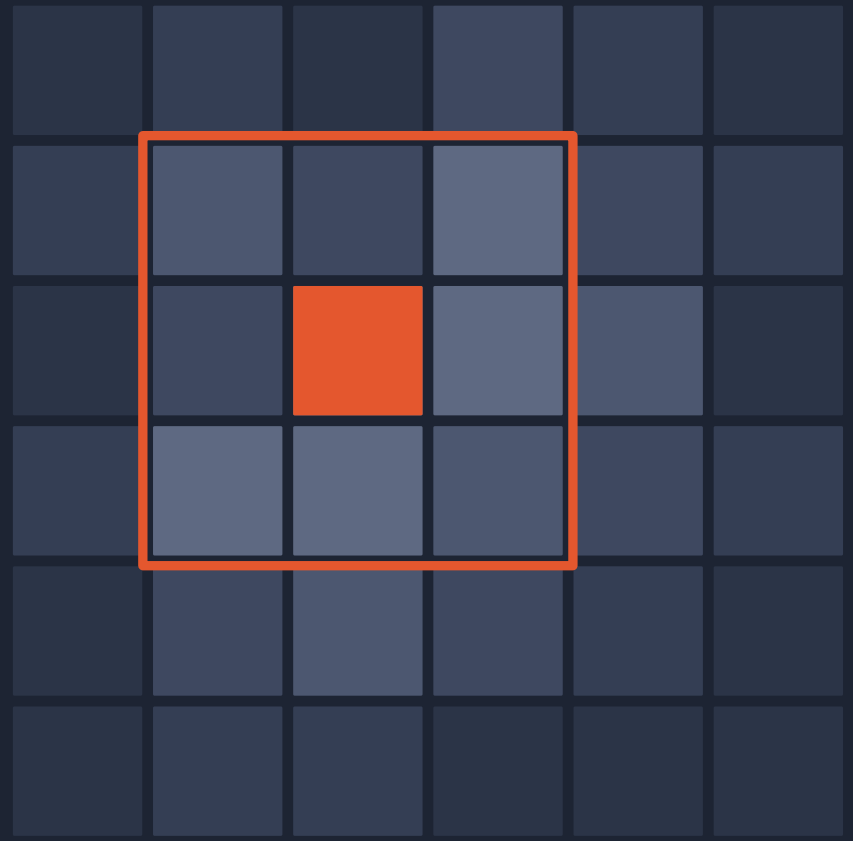
КОМАНДА 22

Сверточные нейронные сети (CNN)

Как нейронная сеть находит признаки на изображении: от операции свёртки до обучения на Fashion-MNIST



Гусев Глеб · Жуков Александр
группа 5030102/30201
Основы машинного обучения · СПбПУ · 2026

Содержание

1 Постановка задачи
зачем нужны CNN и почему не подходит полносвязная сеть

2 Устройство CNN
свёртка, padding и stride, каналы, пулинг, рецептивное поле

3 Численный пример
прямой и обратный проход крошечной CNN вручную, сверка с PyTorch

4 Программная реализация
SimpleCNN на PyTorch: данные, модель, обучение, анализ

5 Альтернативы и сравнение
логистическая регрессия, случайный лес, HOG, MLP, архитектуры



Fashion-MNIST: 70 000 картинок 28×28, 10 классов

Все численные результаты получены программно; код и рисунки — в каталоге src/ (запуск: python run_all.py).



1

Постановка задачи

Какую задачу решают свёрточные сети и почему для изображений нужна особая архитектура

Задача: классификация изображений

картинка
28 × 28 пикселей
оттенки серого

→

модель f(x)
сеть с обучаемыми весами

→

10 вероятностей
p(футболка), p(брюки), …, p(ботинок)

→

ответ
класс с наибольшей вероятностью

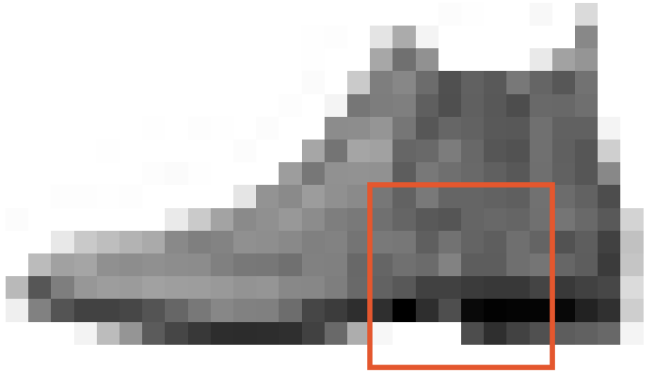


- **Обучение с учителем:** есть 60 000 картинок с правильными метками, нужно подобрать веса так, чтобы модель угадывала метки новых картинок.
- **Набор Fashion-MNIST** (Zalando, 2017): 10 классов одежды, 60 000 обучающих и 10 000 тестовых картинок. Сложнее рукописных цифр MNIST, где почти любая модель даёт 97–98 %.

Нужна модель, которая по сетке пикселей определяет, что изображено, и работает на картинках, которых не видела при обучении.

Изображение — это матрица чисел

28 × 28 = 784 пикселя



фрагмент 8 × 8: яркость 0…255

136	156	134	154	154	156	160	141
143	154	165	170	154	151	154	143
133	136	160	140	155	161	144	155
154	143	148	125	162	162	144	138
156	167	190	190	196	198	198	187
208	255	209	177	245	252	251	251
8	0	0	0	178	208	188	175
0	0	0	0	0	0	0	0

Тест №0 — ботинок; справа увеличен фрагмент у каблука

Чёрно-белое 28×28

1 канал × 28 × 28 = 784 числа от 0 до 255

Цветное 224×224

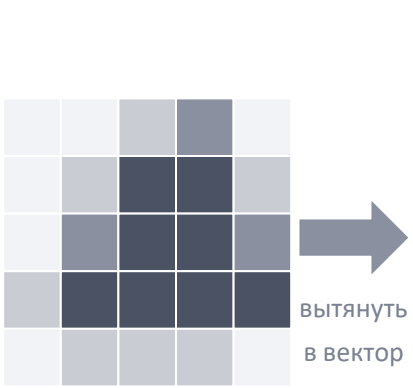
3 канала (R, G, B) × 224 × 224 = 150 528 чисел

Тензор батча N × C × H × W

картинки × каналы × высота × ширина; у нас 128 × 1 × 28 × 28

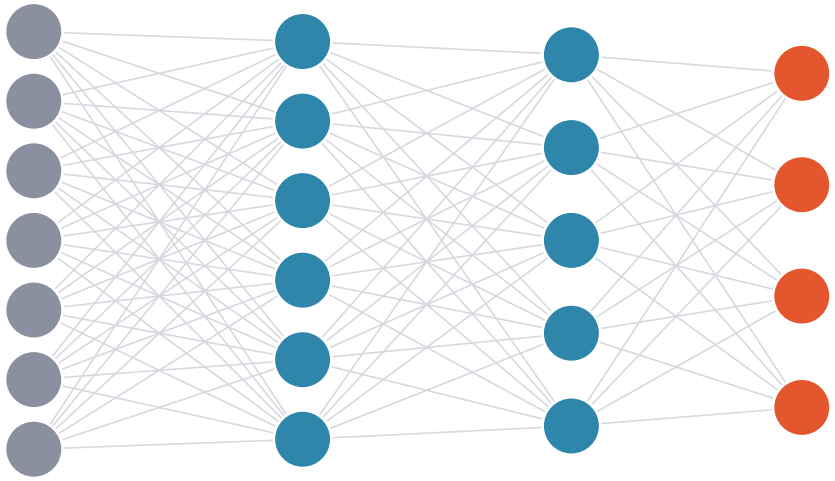
Смысл картинки задаёт взаимное расположение пикселей: соседние пиксели связаны, далёкие — почти нет.

Как решали раньше: полносвязная сеть (MLP)



вытянуть в вектор

картинка
28 × 28



вход
784 числа

слой
256 нейронов

слой
128 нейронов

выход
10 классов

каждый нейрон слоя соединён со всеми входами — отсюда «полносвязная»

Один нейрон

$$y = f(w_1x_1 + w_2x_2 + \ldots + w_nx_n + b)$$

w — веса, b — смещение (их подбирает обучение); f — функция активации, например ReLU(z) = max(0, z)

Слой	Параметров
784 → 256	784·256 + 256 = 200 960
256 → 128	32 896
128 → 10	1 290
Всего	**235 146**

89,0 %

точность MLP на Fashion-MNIST — сеть работает

MLP видит картинку как список из 784 чисел и не знает, какие пиксели соседние. Отсюда две проблемы.

Полносвязная сеть: слишком много параметров

Полносвязный (Linear) слой соединяет каждый вход с каждым нейроном: число весов = входы × нейроны.

Вход → выход	Свёртка 3×3	Полносвязный, 1000 нейронов	Полносвязный, тот же размер выхода
1×28×28 → 32 карты 28×28	**320**	785 000	19 694 080
3×224×224 → 64 карты 224×224	**1 792**	150 529 000	483 388 358 656

150 млн

весов у одного полносвязного слоя на 1000 нейронов для фото 224×224

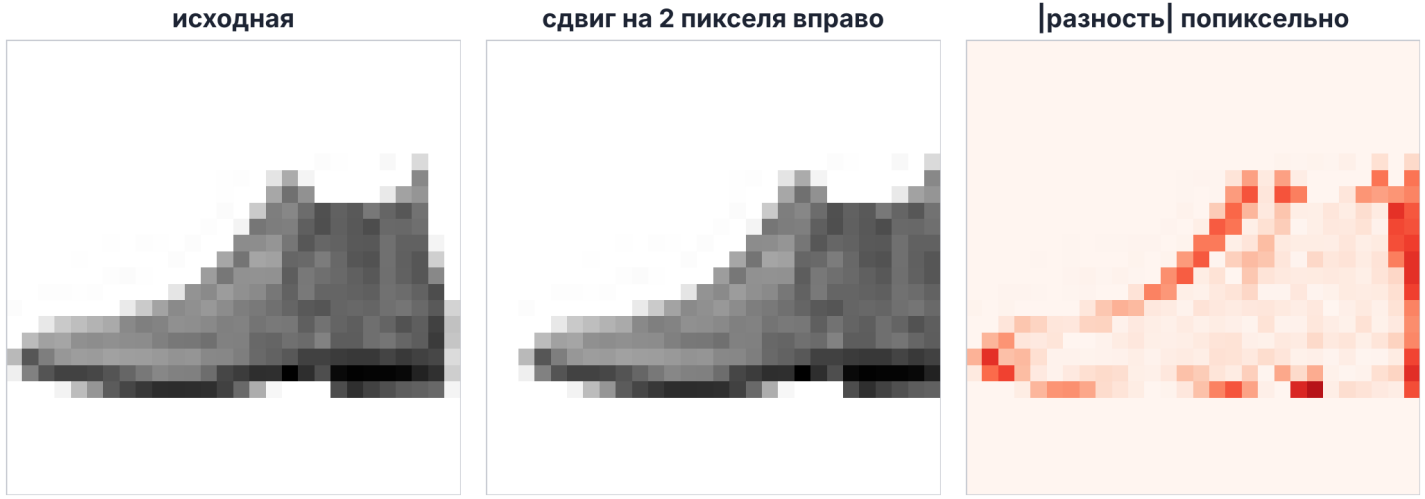
1 792

весов у свёрточного слоя с 64 фильтрами 3×3 — для картинки любого размера

- Много весов — нужно много данных, иначе модель запоминает обучающую выборку (переобучение).
- Каждый вес полносвязного слоя «привязан» к конкретному пикселю — знание не переносится между частями картинки.

Свёрточный слой использует одно маленькое ядро во всех позициях, поэтому весов в тысячи раз меньше.

Полносвязная сеть: сдвиг меняет всё



Сдвиг	Сходство векторов (косинус)	Пикселей изменилось > 25 %
1 пикс.	0,89	12 %
2 пикс.	0,80	18 %
3 пикс.	0,74	23 %
4 пикс.	0,68	27 %
другая вещь того же класса	**0,75**	—

Средние по 10 000 тестовым картинкам

- Полносвязная сеть сравнивает вход **i** с весом **i**: сдвиг на пару пикселей для неё — другая картинка.
- Уже при сдвиге на 3 пикселя вектор ботинка «дальше» от исходного, чем в среднем **другой ботинок**.

Нужна архитектура, которая ищет признак (край, каблук, воротник) в любом месте картинки, а не в фиксированном пикселе.

Три идеи свёрточной сети



Локальность

Нейрон смотрит не на всю картинку, а на маленькое окно (например, 3×3). Края и текстуры — локальные признаки.



Общие веса

Одно и то же ядро скользит по всей картинке. Признак, выученный в одном месте, находится в любом другом.



Иерархия признаков

Слои складываются: края → части (каблук, рукав) → целые объекты. С глубиной нейрон «видит» всё большую область.

История

1980

Неокогнитрон (К. Фукусима)

1989–1998

LeNet: обучение свёрток обратным распространением (Я. ЛеКун), чтение чеков

2012

AlexNet: ImageNet, ошибка top-5 15,3 % против 26,2 % у лучшего не-нейросетевого

2015

ResNet: до 152 слоёв, ошибка top-5 3,6 % (ансамбль)

2020–2022

ViT и ConvNeXt: трансформеры и обновлённые CNN



2

Устройство CNN

Из каких операций собрана свёрточная сеть и как они преобразуют тензор

Операция свёртки



$$S(i,j) = \Sigma\,\Sigma\, X(i+a, j+b) \cdot K(a,b) + b_0$$

X — вход, K — ядро (фильтр), b_0 — смещение; сумма по a и b — по всем клеткам окна

```python
02_convolution_basics.py · свёртка на NumPy

def conv2d(x, kernel, stride=1, padding=0):
    x = np.pad(x, padding)                    # рамка из нулей
    k = kernel.shape[0]
    h = (x.shape[0] - k) // stride + 1        # ⌊(N + 2P − K) / S⌋ + 1
    w = (x.shape[1] - k) // stride + 1
    out = np.empty((h, w))
    for i in range(h):
        for j in range(w):
            window = x[i*stride : i*stride + k, j*stride : j*stride + k]
            out[i, j] = np.sum(window * kernel)
    return out                    # max |conv2d − F.conv2d| = 7·10⁻¹⁵
```

- Окно размером с ядро ставится на вход, клетки попарно перемножаются и складываются — получается одно число.
- Окно сдвигается на одну клетку, и расчёт повторяется. Все числа вместе — карта признаков (feature map).
- На рисунке: 9 произведений в оранжевой рамке дают −2.

- Строго говоря, это взаимная корреляция (ядро не переворачивается), но в глубоком обучении её называют свёрткой.

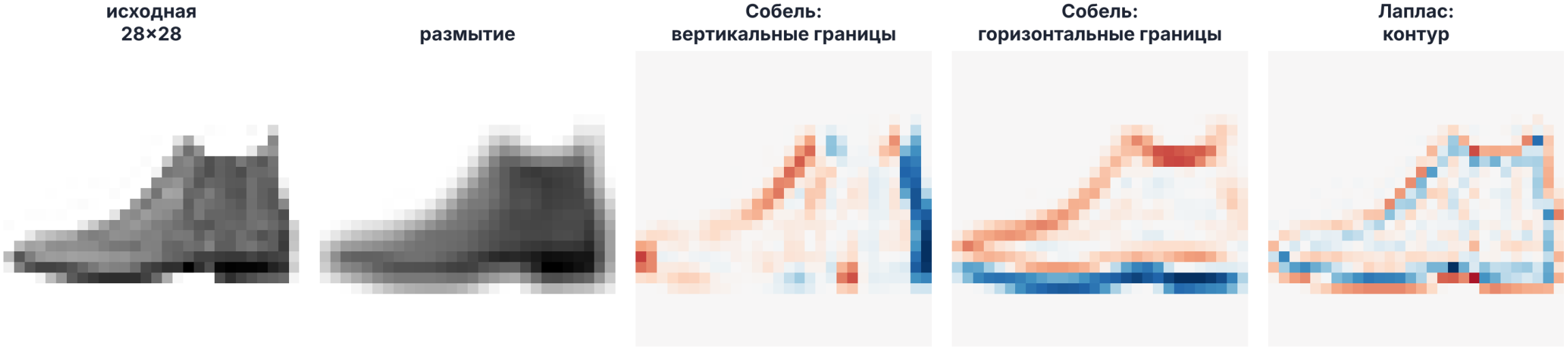
Ядро — детектор признака

0.11	0.11	0.11
0.11	0.11	0.11
0.11	0.11	0.11

-1	0	+1
-2	0	+2
-1	0	+1

-1	-2	-1
0	0	0
+1	+2	+1

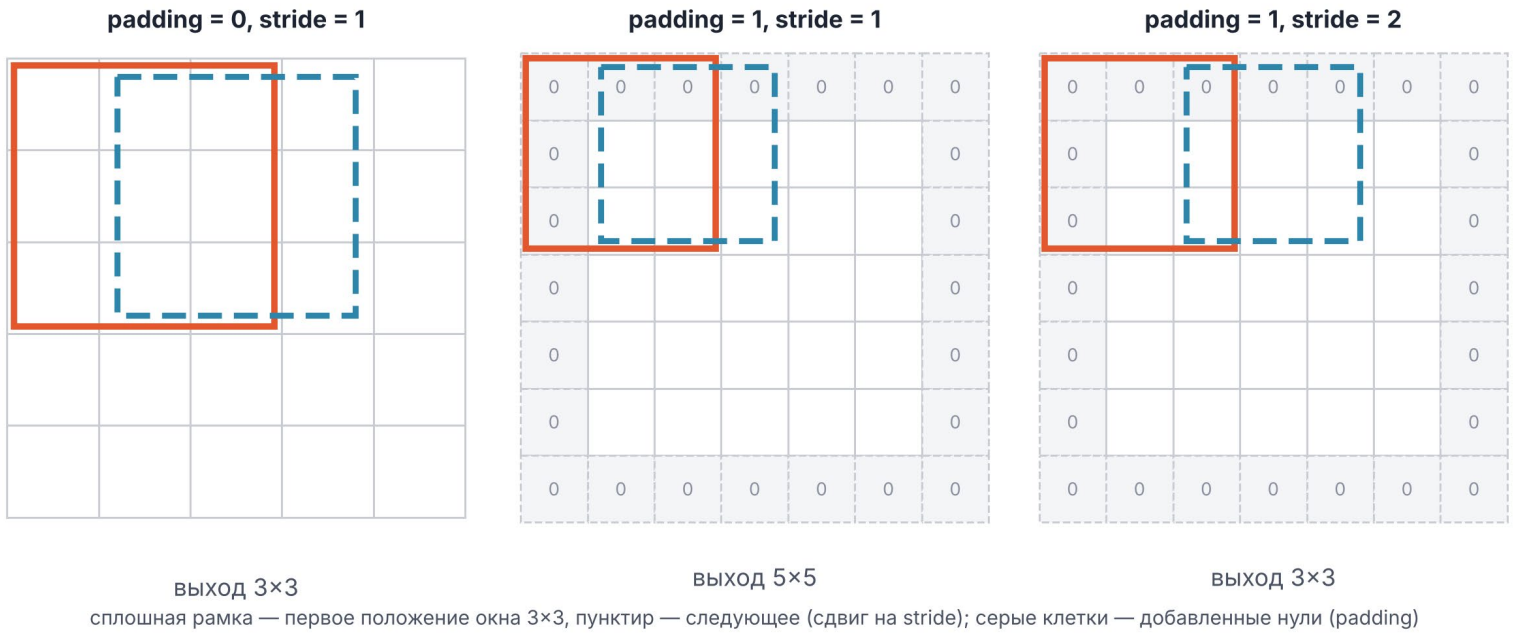
0	+1	0
+1	-4	+1
0	+1	0



Верхний ряд — ядра 3×3, нижний — результат свёртки ботинка с каждым ядром (padding = 1). Красное — положительный отклик, синее — отрицательный.

В CNN значения ядер не задаются вручную, как здесь, а подбираются градиентным спуском — сеть сама решает, какие признаки полезны.

Отступ (padding) и шаг (stride)



сплошная рамка — первое положение окна 3×3, пунктир — следующее (сдвиг на stride); серые клетки — добавленные нули (padding)

$$O = \lfloor (N + 2P - K) / S \rfloor + 1$$

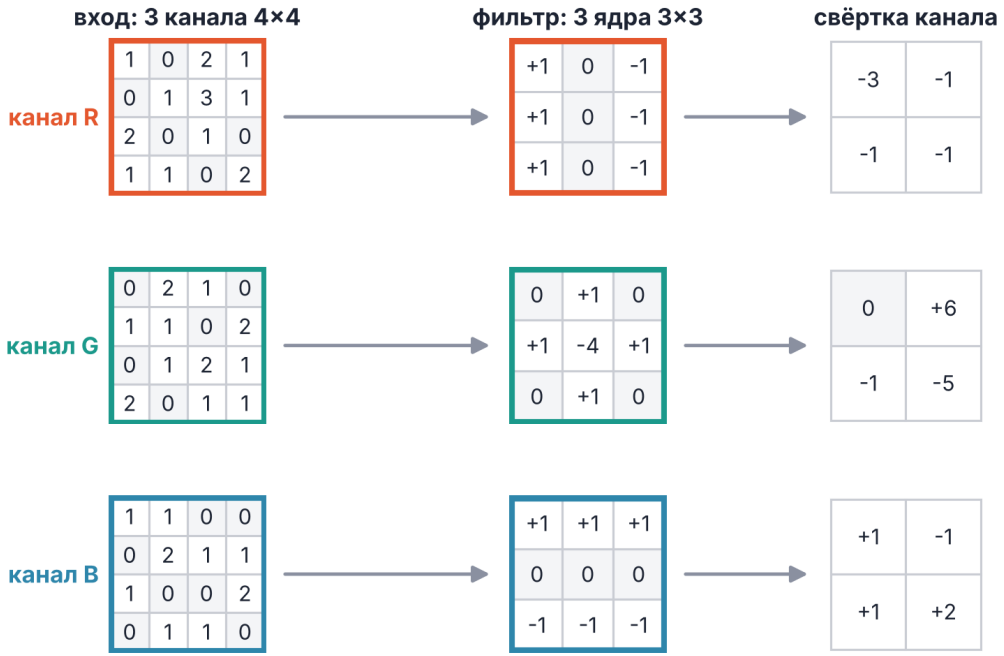
N — размер входа, K — ядра, P — отступ, S — шаг

- **Padding P** — рамка из нулей. При K = 3, P = 1 размер сохраняется, и края картинки учитываются наравне с центром.
- **Stride S** — на сколько клеток сдвигается окно. S = 2 уменьшает карту вдвое.
- **Проверка:** `(5 + 2 − 3) / 2 + 1 = 3`

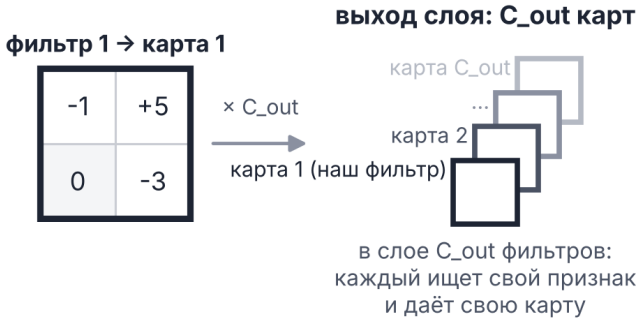
Вход 28×28: K, P, S	3, 0, 1	3, 1, 1	5, 2, 1	3, 1, 2	7, 3, 2
Расчёт ⌊(28 + 2P − K) / S⌋ + 1	25 + 1	27 + 1	27 + 1	13 + 1	13 + 1
Выход	26×26	28×28	28×28	14×14	14×14

В SimpleCNN свёртки идут с K = 3, P = 1, S = 1 (размер сохраняется), а уменьшает карты пулинг.

Несколько каналов и число параметров



сумма
3 карт
+ b (+1)



Один фильтр даёт одну карту. Слой из C_out фильтров даёт C_out карт — как на слайде с ядрами: 4 ядра → 4 картинки.

1. Ядро K×K: K^2 весов. **2. Фильтр = C_in ядер + 1 смещение:** $C_{in} \cdot K^2 + 1$

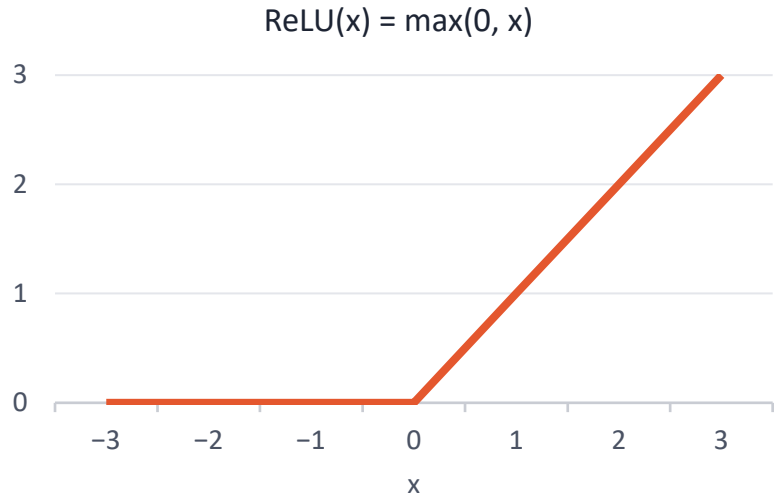
3. Слой = C_out фильтров: **params = $C_{out} \cdot (C_{in} \cdot K^2 + 1)$**

На рисунке: 3·9 + 1 = 28 параметров на один фильтр

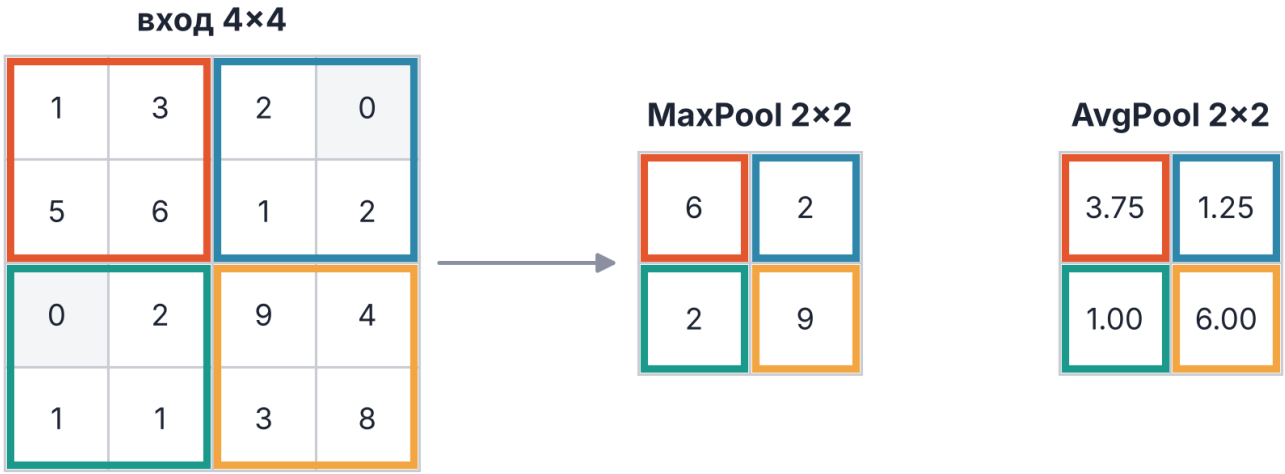
Слой SimpleCNN	Расчёт	Параметров
Conv 1 → 32	32 · (1·9 + 1)	**320**
Conv 32 → 64	64 · (32·9 + 1)	**18 496**
Linear 3136 → 128	3136 · 128 + 128	**401 536**

В нашей сети 95 % параметров — в полносвязном слое; свёрточная часть весит 19 тыс. параметров из 422 тыс.

Нелинейность и пулинг



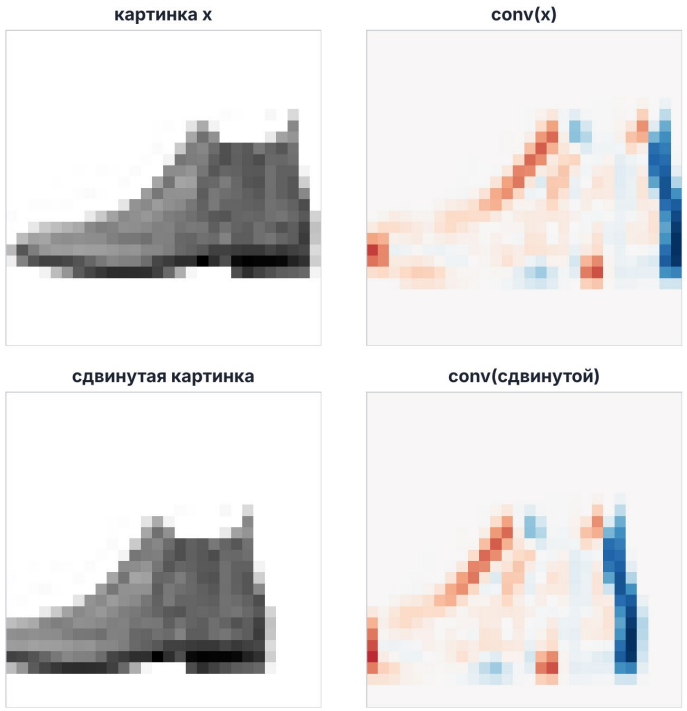
Без нелинейности две свёртки подряд — снова одна линейная операция, и глубина ничего бы не давала.



- **MaxPool 2×2:** максимум в каждом квадрате 2×2 — карта уменьшается вдвое, сильный отклик сохраняется, мелкие сдвиги почти не меняют результат.
- **AvgPool:** среднее вместо максимума. **Global Average Pooling** усредняет всю карту в одно число — так устроены ResNet и MobileNet.
- **Параметров у пулинга нет** — он ничего не обучает.

Типичный блок CNN: свёртка → нормализация → ReLU → пулинг. Несколько блоков подряд, затем классификатор.

Сдвиг и рецептивное поле



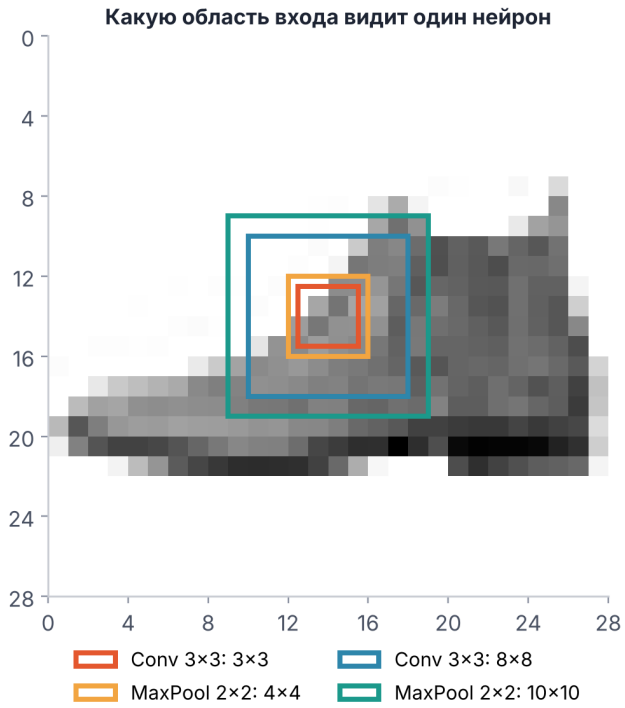
Эквивариантность к сдвигу

Сдвинули картинку — карта признаков сдвинулась так же:

`conv(shift(x)) = shift(conv(x))`

Проверка на ботинке, сдвиг (3, −4): максимальная разница во внутренней области = 0.

Пулинг и усреднение превращают это в приблизительную инвариантность: ответ сети мало зависит от точного положения.



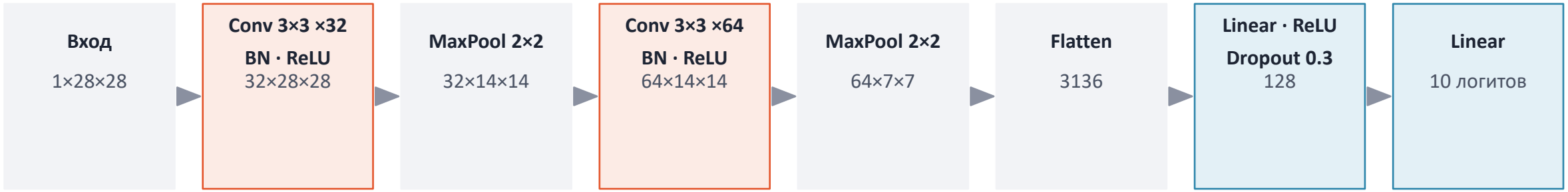
Рецептивное поле — область входа, от которой зависит нейрон: **Conv 3×3 → 3×3, MaxPool 2×2 → 4×4, Conv 3×3 → 8×8, MaxPool 2×2 → 10×10**

`r ← r + (K − 1) · j,  j ← j · S` (j — за расстояние между соответствующими областями соседних элементов текущего слоя)

K – размер окна на шаге; S – stride - шаг

После двух блоков нейрон «видит» 10×10 пикселей — это уже часть вещи, а не отдельный край.

Архитектура нашей сети SimpleCNN

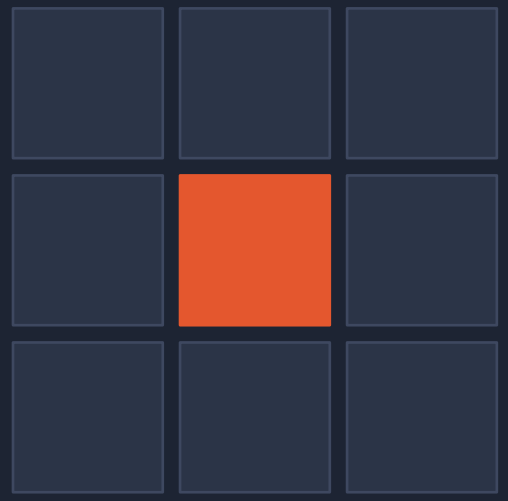


models.py

```python
class SimpleCNN(nn.Module):
    def __init__(self, n_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),    # 32×28×28
            nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2),                               # 32×14×14
            nn.Conv2d(32, 64, kernel_size=3, padding=1),   # 64×14×14
            nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2),                               # 64×7×7
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),                                  # 3136
            nn.Linear(64 * 7 * 7, 128), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(128, n_classes),                     # 10 логитов
```

- **BatchNorm (BN)** — нормирует каждую карту по мини-батчу: обучение быстрее и стабильнее.
- **Dropout** — при обучении случайно обнуляет 30 % нейронов: меньше переобучения.
- **Логиты** — оценки классов до softmax; softmax превращает их в вероятности.
- **Параметров: 421 834**, из них в свёртках — 19 008 (4,5 %).

Свёрточная часть превращает 784 пикселя в 64 карты признаков 7×7, классификатор по ним выбирает класс.



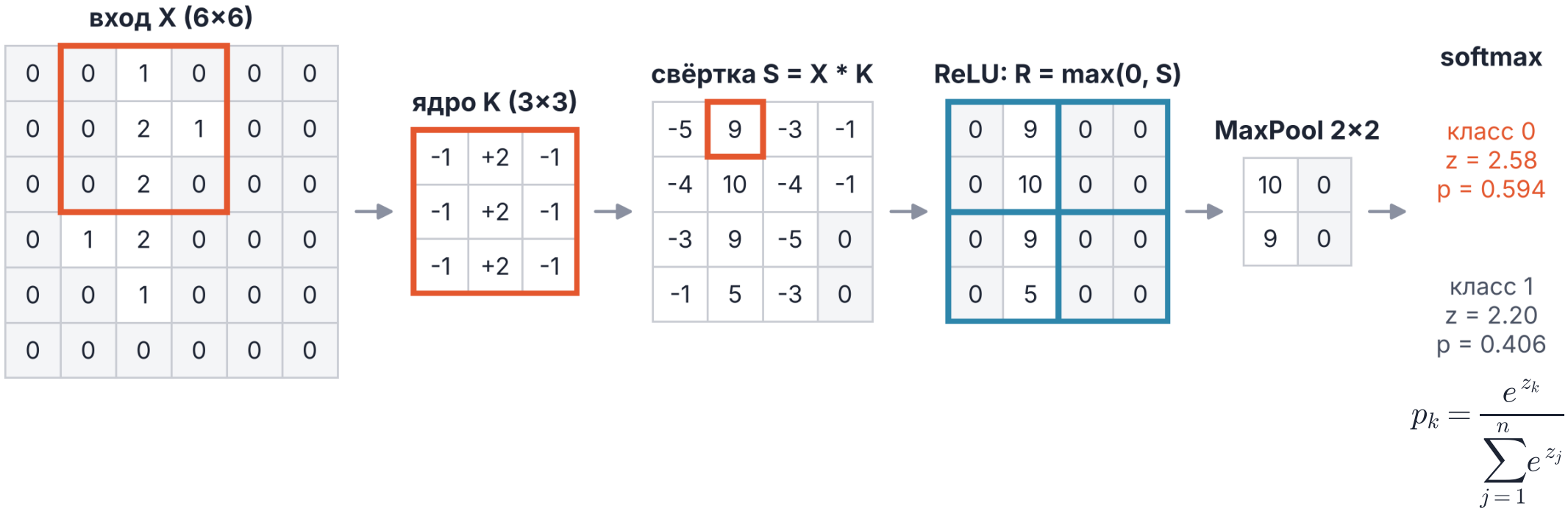
3

Численный пример

Один шаг обучения крошечной свёрточной сети вручную и сверка с PyTorch

Постановка и шаг 1: свёртка

Сеть: X (6×6) → свёртка с K (3×3) → ReLU → MaxPool 2×2 → вектор f (4) → z = W·f + b (2 класса) → softmax → кросс-энтропия. **Класс 0 — «верт**



$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{n} e^{z_j}}$$

S[0, 1] = (−1)·0 + 2·1 + (−1)·0 + (−1)·0 + 2·2 + (−1)·1 + (−1)·0 + 2·2 + (−1)·0 = **9**

размер выхода: (6 − 3) / 1 + 1 = 4

Оранжевые рамки: окно на входе и посчитанная клетка S[0, 1]; синие рамки — квадраты пулинга 2×2

Шаги 2–5: ReLU, пулинг, полносвязный слой, softmax

Шаг 2. ReLU

R = max(0, S)

отрицательные отклики обнуляются

Шаг 3. MaxPool 2×2

```
P = [[10, 0],
     [9, 0]]
f = (10, 0, 9, 0)
```

Шаг 4. z = W·f + b

$z_0 = 0{,}15 \cdot 10 + 0{,}12 \cdot 9 = 2{,}58$

$z_1 = 0{,}1 \cdot 10 + 0{,}1 \cdot 9 + 0{,}3 = 2{,}20$

Шаг 5. softmax и потеря

$p = (0{,}594;\ 0{,}406)$

$L = -\ln p_0 = 0{,}521$

Параметр	Значение
W (2×4)	[[0,15; −0,10; 0,12; 0,05], [0,10; 0,10; 0,10; −0,05]]
b	(0; 0,3)
Шаг обучения	0,01

softmax: $p_k = \exp(z_k) / \Sigma_j \exp(z_j)$

кросс-энтропия: L = −ln p_y, где p_y – вероятность правильного варианта

Сеть уже склоняется к верному классу ($p_0 \approx 0{,}59$), но не уверена — обучение должно увеличить p_0.

Прямой проход — это несколько десятков умножений и сложений; сложность CNN — в количестве, а не в операциях.

Шаг 6: обратный проход (правило цепочки)

$\partial L/\partial z = p - y$

$(-0{,}4061;\ 0{,}4061)$

где $y = (1, 0)$ – правильный ответ, записанный вектором

$\partial L/\partial f = W^T \cdot \partial L/\partial z$

$-0{,}0203;\ 0{,}0812;$
$-0{,}0081;\ -0{,}0406$

MaxPool

градиент идёт только в позицию максимума

ReLU

пропускает градиент только там, где $S > 0$

$\partial L/\partial K = X * \partial L/\partial S$

свёртка входа с градиентом карты

$\partial L/\partial S$

0	0	0	0
0	-0.0203	0	0
0	-0.0081	0	0
0	0	0	0

≠ 0 только 2 клетки:
MaxPool и ReLU обнулили остальные

окно X под S[1,1]

0	2	1
0	2	0
1	2	0

× (-0.0203)

$\partial S[i,j]/\partial K$ = окно X, которое ядро накрывало для S[i,j]

+

окно X под S[2,1]

0	2	0
1	2	0
0	1	0

× (-0.0081)

ядро общее → вклады всех клеток складываются

=

$\partial L/\partial K$

0	-0.057	-0.020
-0.008	-0.057	0
-0.020	-0.049	0

$\partial L/\partial b_K = \Sigma\ \partial L/\partial S = -0.0284$

- Градиент по ядру — снова свёртка: та же операция работает и вперёд, и назад.
- Знаки $\partial L/\partial K$ отрицательны в центральном столбце: шаг **усиливает** детектор вертикальной линии.
- Градиент по K мал, потому что малы веса W; полносвязный слой меняется сильнее.

Обновление: $K \leftarrow K - 0{,}01 \cdot \partial L/\partial K$, аналогично для b_K, W и b. $\partial L/\partial b_K = \Sigma\ \partial L/\partial S = -0{,}0284$.

Шаг 7: результат и сверка с PyTorch

0,594 → 0,865

вероятность верного класса после одного шага

0,521 → 0,145

значение функции потерь

Ядро после шага (центральный столбец вырос):

−1,0000	+2,0006	−0,9998
−0,9999	+2,0006	−1,0000
−0,9998	+2,0005	−1,0000

```
03_numerical_example.py · те же вычисления в PyTorch

x = torch.tensor(X).view(1, 1, 6, 6)
k = torch.tensor(K, requires_grad=True)
w = torch.tensor(W, requires_grad=True)
b = torch.tensor(B, requires_grad=True)

s = F.conv2d(x, k.view(1, 1, 3, 3))       # свёртка
f = F.max_pool2d(F.relu(s), 2).flatten()  # ReLU + MaxPool
z = w @ f + b                             # полносвязный слой
loss = F.cross_entropy(z.view(1, -1), torch.tensor([0]))
loss.backward()                           # autograd

np.abs(k.grad.numpy() - dK_manual).max()  # 8.3e-17
```

Величина	S	L	∂L/∂K	∂L/∂W	∂L/∂b
max |ручной − PyTorch|	0	$1 \cdot 10^{-16}$	$8 \cdot 10^{-17}$	$9 \cdot 10^{-16}$	$1 \cdot 10^{-16}$

Ручной расчёт совпал с автоматическим дифференцированием PyTorch до ошибок округления (~10^{-16}).



4

Программная реализация

SimpleCNN на PyTorch: данные, модель, цикл обучения, результаты и их анализ

Данные: загрузка и подготовка

data.py (упрощённо)

```
transform = transforms.Compose([
    transforms.ToTensor(),                      # [0, 255] → [0, 1]
    transforms.Normalize((0.2860,), (0.3530,)), # (x − mean) / std
])
full = datasets.FashionMNIST("data", train=True,
                             download=True, transform=transform)
test = datasets.FashionMNIST("data", train=False,
                             download=True, transform=transform)
idx = np.random.RandomState(42).permutation(60_000)
train = Subset(full, idx[5_000:])               # 55 000
val = Subset(full, idx[:5_000])                 # 5 000

train_loader = DataLoader(train, batch_size=128, shuffle=True)
images, labels = next(iter(train_loader))
images.shape        # torch.Size([128, 1, 28, 28])
```

Первый запуск сам скачивает архивы (~30 МБ) в src/data/.

55 000
обучение

5 000
валидация

10 000
тест

128
картинок в мини-батче

- **Нормализация:** после неё яркости в среднем 0 с разбросом 1 — градиентному спуску проще.
- **Валидация:** по ней выбираем лучшую эпоху; тест используем один раз, в конце.
- **Мини-батч:** порция картинок для одного шага градиентного спуска.

Три непересекающиеся выборки: на одной учимся, по второй настраиваемся, на третьей честно оцениваем.

Цикл обучения

engine.py (упрощённо) · 04_train_cnn.py

```
model = SimpleCNN().to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
scheduler = CosineAnnealingLR(optimizer, T_max=12)  # шаг: 1e-3 → 0
loss_fn = nn.CrossEntropyLoss()        # softmax + кросс-энтропия

for epoch in range(12):
    model.train()                      # режим обучения
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        logits = model(images)         # 1. прямой проход
        loss = loss_fn(logits, labels) # 2. функция потерь
        optimizer.zero_grad()
        loss.backward()                # 3. обратный проход
        optimizer.step()               # 4. шаг оптимизатора
    scheduler.step()                   # уменьшаем шаг обучения
    val_loss, val_acc = evaluate(model, val_loader, device)
    # запоминаем веса эпохи с лучшей val_acc
```

1

Прямой проход

сеть выдаёт 10 логитов для каждой картинки батча

2

Функция потерь

кросс-энтропия между softmax(логиты) и меткой

3

Обратный проход

loss.backward() считает ∂L/∂w для всех 422 тыс. весов

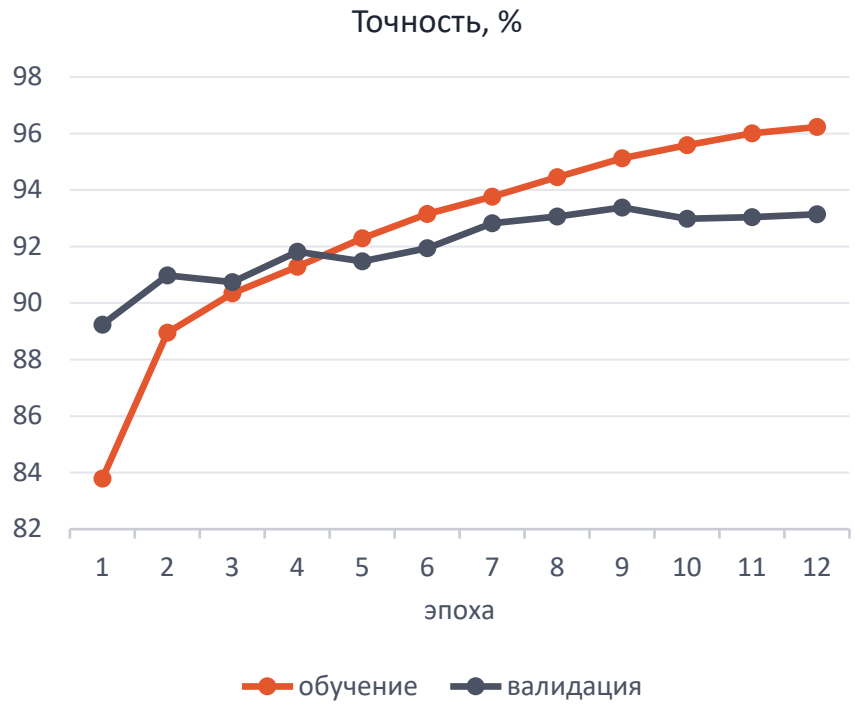
4

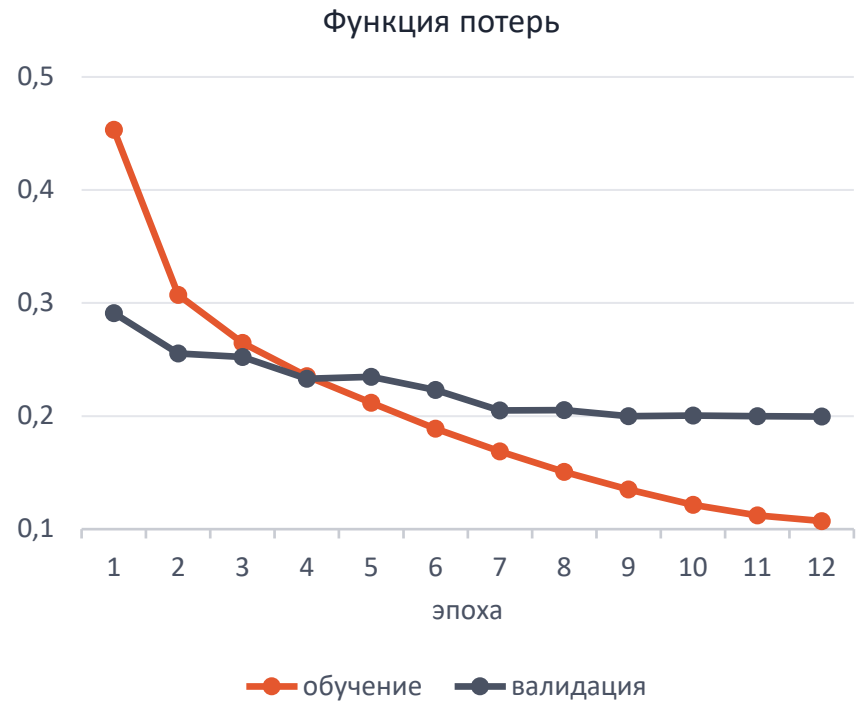
Шаг Adam

сдвиг весов против градиента; размер шага плавно уменьшается к концу обучения

Эпоха = проход по всем 55 000 картинкам = 430 шагов. 12 эпох на процессоре — около 7 минут, на видеокарте — меньше минуты.

Результат обучения





92,52 %

точность на тесте (9 252 из 10 000)

9

лучшая эпоха по валидации

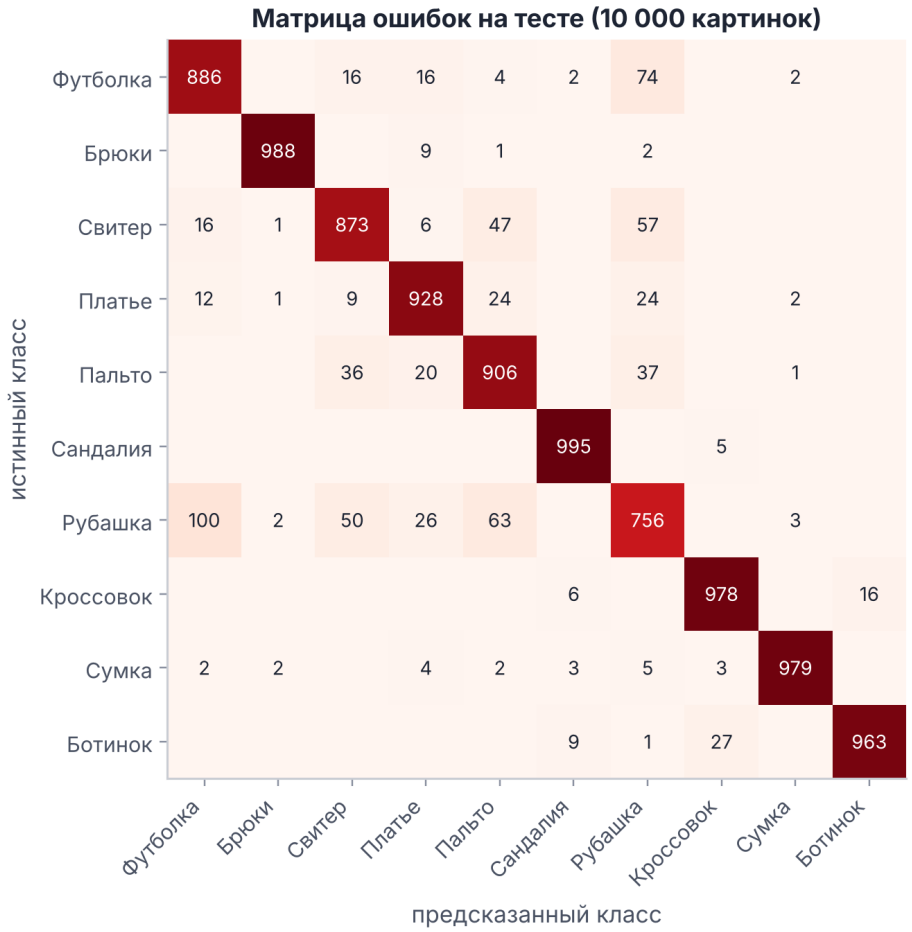
7 мин

обучение на процессоре (2 потока)

- Точность на обучении продолжает расти, на валидации выходит на плато ~92–93 %: разрыв к 12-й эпохе — 3,1 п.п., начинается переобучение.
- Поэтому сохраняются веса лучшей эпохи по валидации, а не последней.

Небольшая CNN из двух свёрточных блоков правильно классифицирует 92 % незнакомых картинок.

Где сеть ошибается



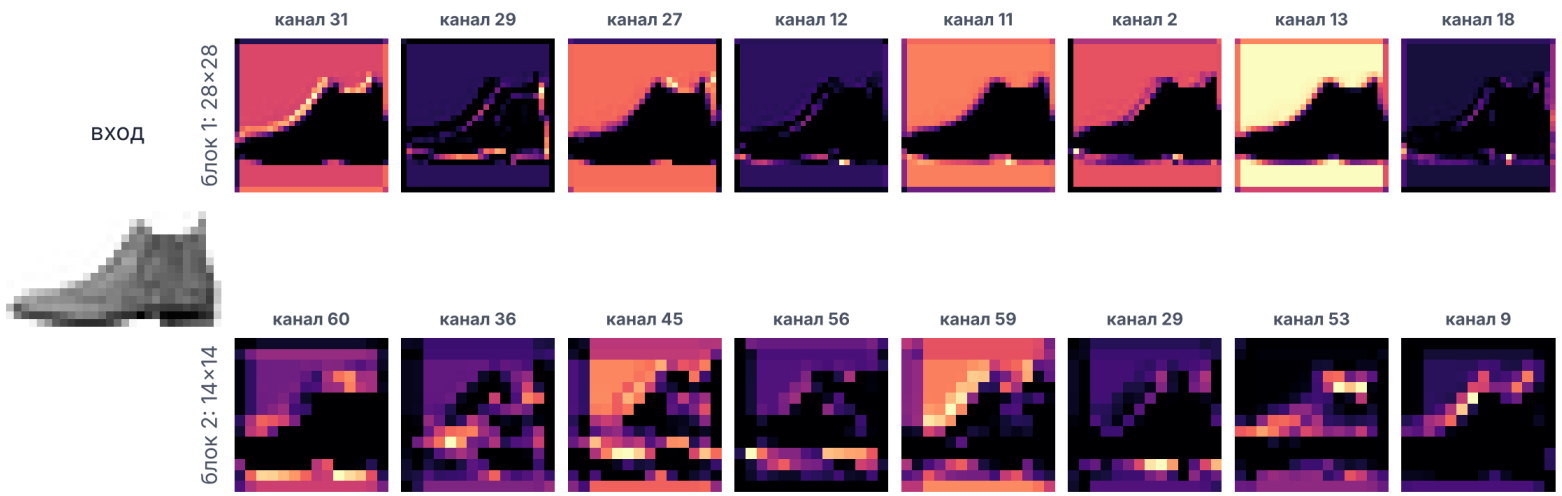
Класс	Точность
Рубашка	75,6 %
Свитер	87,3 %
Футболка	88,6 %
…	…
Брюки	98,8 %
Сандалия	99,5 %

- **Сложнее всего — верх одежды:** рубашка → футболка (100), футболка → рубашка (74), рубашка → пальто (63).
- **Обувь и сумки** различаются почти без ошибок: у них характерный силуэт.

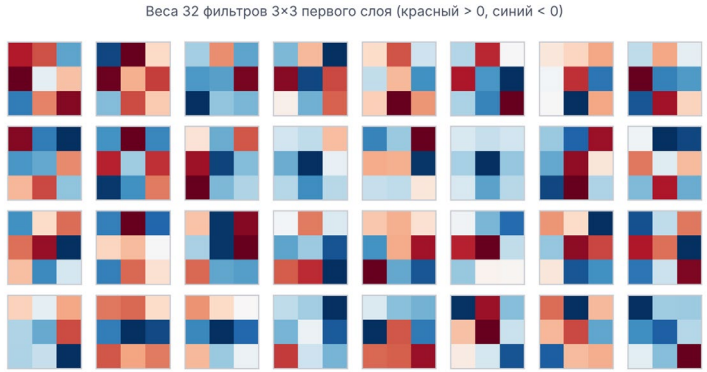


Всего 748 ошибок из 10 000; многие «ошибки» спорны и для человека при разрешении 28×28.

Что выучила сеть



Карты признаков ботинка после первого (28×28) и второго (14×14) блоков: 8 самых активных каналов из 32 и 64

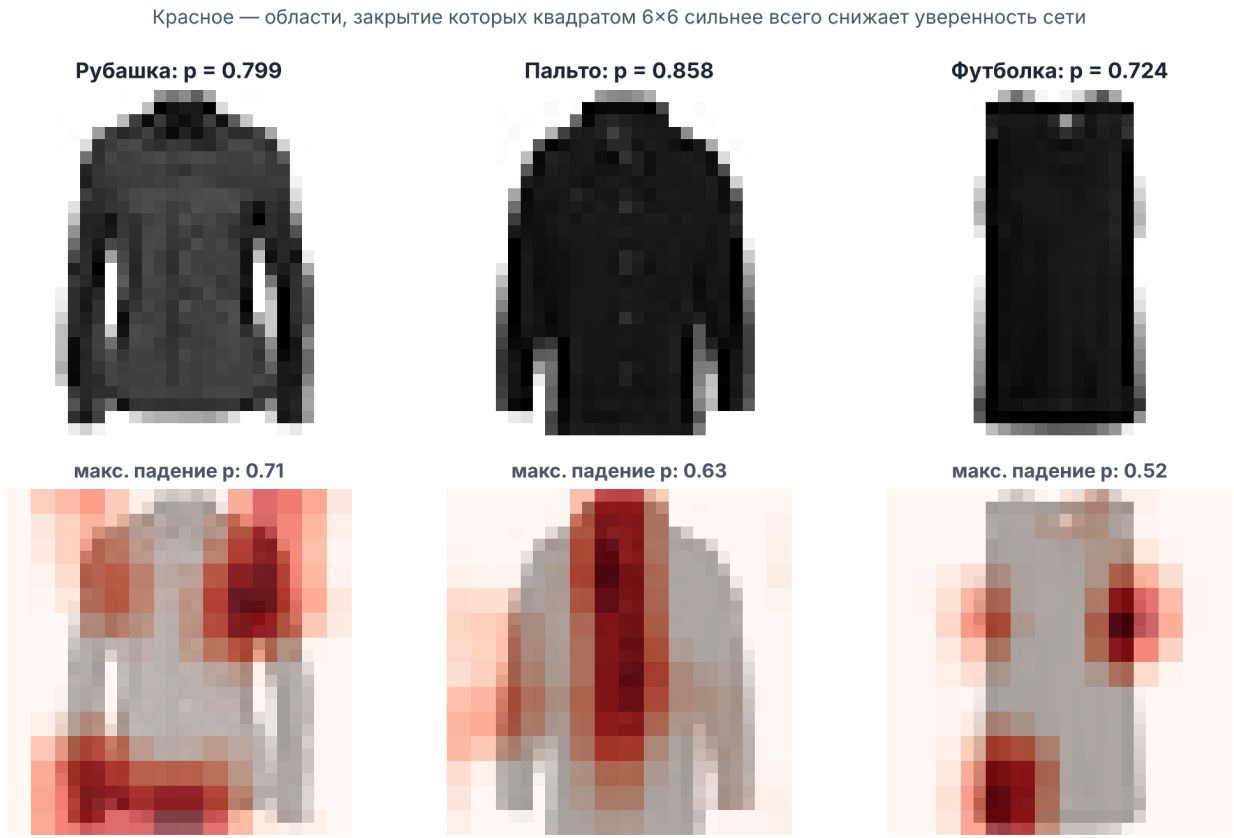


Веса 32 фильтров первого слоя — сеть сама получила аналоги детекторов границ

- **Блок 1** реагирует на простые вещи: контур, верхний край, подошву, фон.
- **Блок 2** комбинирует их: отдельные карты «загораются» на голенище, каблуке, носке.
- **Фильтры первого слоя** — среди них есть детекторы границ разных направлений, как ядра Собеля; их никто не задавал.

Иерархия признаков возникает сама: от краёв на первом слое к частям вещи на втором.

Куда смотрит сеть: карта перекрытия



Метод перекрытия (occlusion)

- Закрываем квадрат 6×6 цветом фона и смотрим, насколько упала вероятность верного класса.
- Двигаем квадрат с шагом 2 по всей картинке и усредняем падение в каждом пикселе.
- Красное — области, без которых сеть перестаёт узнавать вещь.

Наблюдения

- **Пальто:** застёжка по центру сверху донизу.
- **Топ (класс «футболка»):** проймы по бокам — то, чем он отличается от платья и рубашки.
- **Рубашка:** плечо и нижний край.
- Взяты картинки с уверенностью 0,6–0,9: при p ≈ 1 закрытие маленького участка ничего не меняет.

Сеть опирается на осмысленные части вещи, а не на случайные пиксели фона.

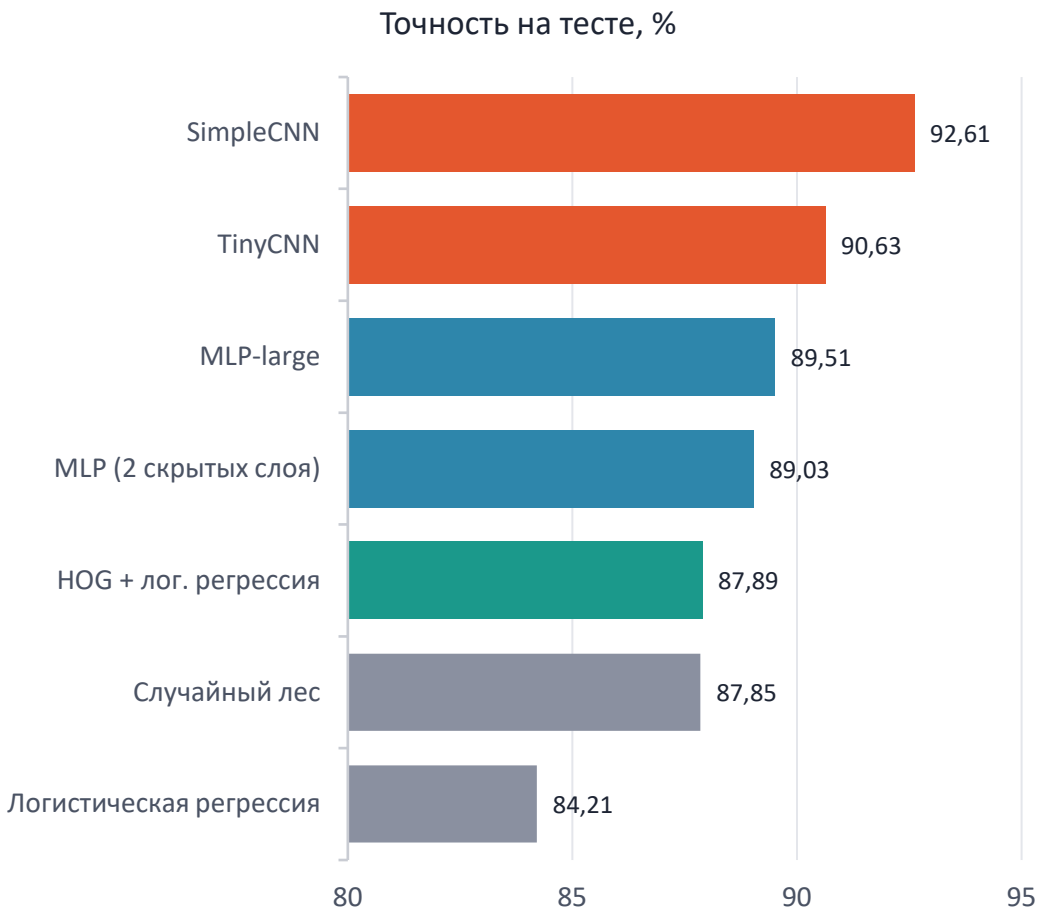


5

Альтернативы и сравнение

Классические методы, ручные признаки, полносвязные сети, современные архитектуры

Сравнение на Fashion-MNIST

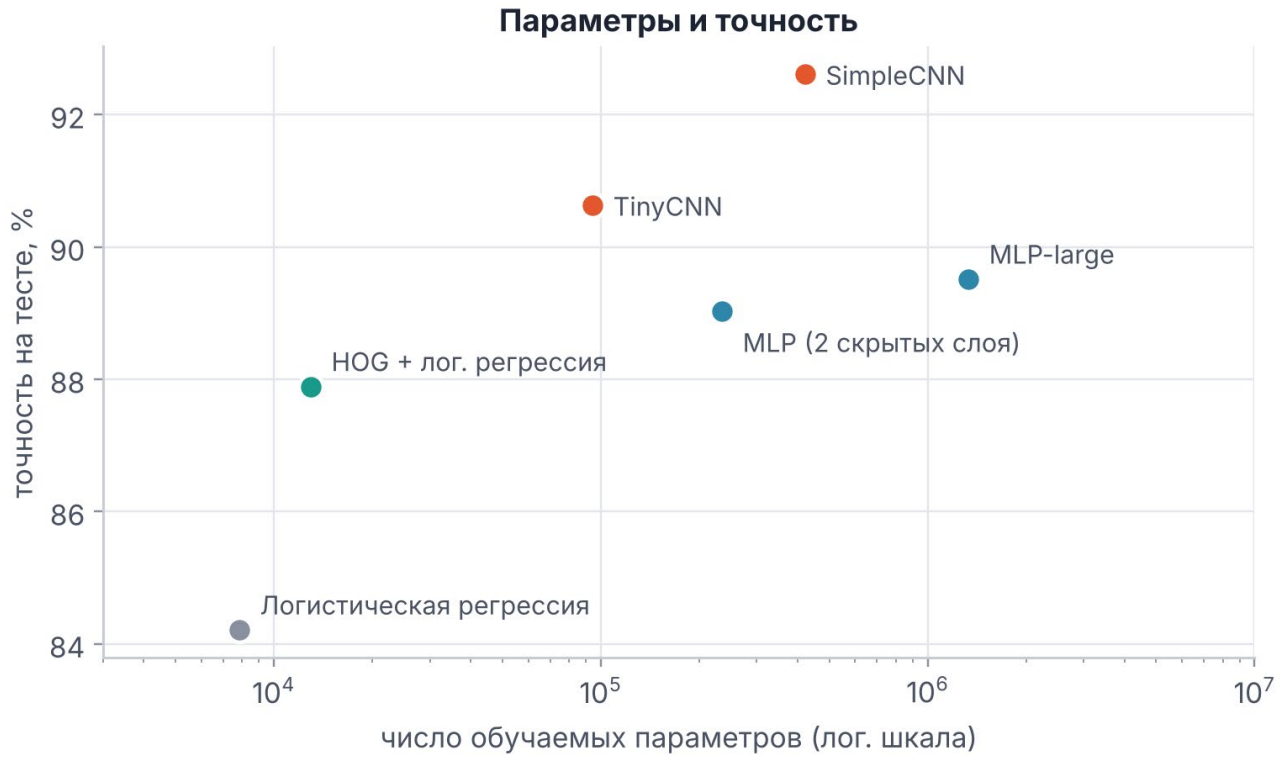


Модель	Точность, %	Параметров	Обучение
SimpleCNN	**92,61 ± 0,19**	421 834	7,4 мин
TinyCNN	**90,63 ± 0,18**	94 410	9,2 мин
MLP-large	89,51 ± 0,12	1 333 770	2,3 мин
MLP (2 скрытых слоя)	89,03 ± 0,03	235 146	88 с
HOG + лог. регрессия	87,89	12 970	1,7 мин
Случайный лес	87,85	—	74 с
Логистическая регрессия	84,21	7 850	1,8 мин

Нейросети: среднее ± ст. отклонение по 3 запускам (seed), 12 эпох. Время — процессор, 2 потока; для HOG включает расчёт признаков. Параметры случайного леса не сравнимы с весами (200 деревьев).

Обе CNN лучше всех альтернатив; TinyCNN с 94 410 параметрами точнее MLP-large с 1,33 млн.

Дело не в числе параметров



Полносвязные сети

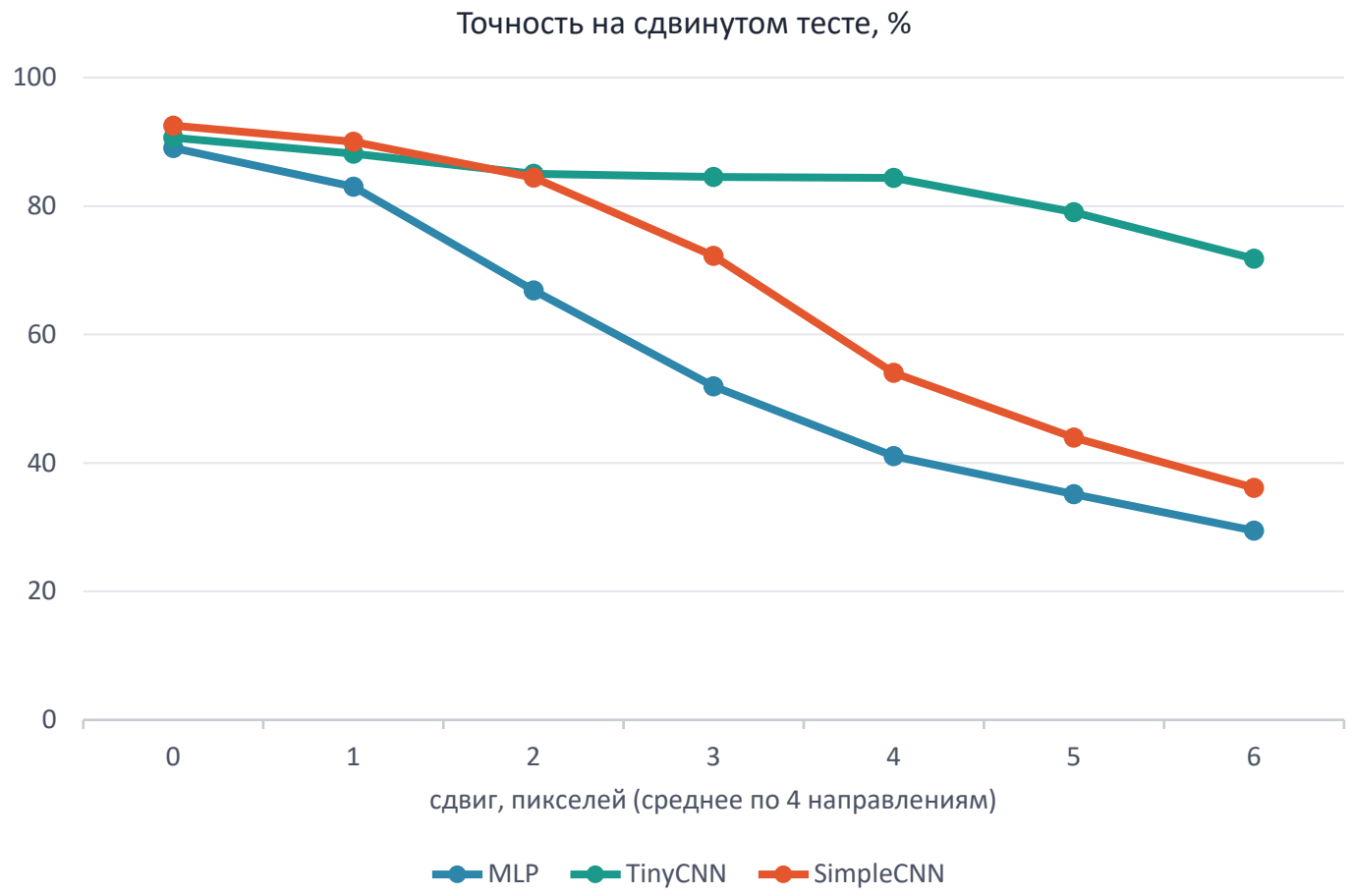
- MLP: 235 146 параметров → 89,03 %
- MLP-large: в 6 раз больше → 89,51 %

Свёрточные сети

- TinyCNN: 94 410 параметров → 90,63 %
- SimpleCNN: 421 834 → 92,61 %
- TinyCNN вместо большого Linear использует Global Average Pooling

Увеличение MLP почти не помогает: выигрыш CNN даёт архитектура — локальность и общие веса, — а не размер.

Эксперимент 1: устойчивость к сдвигу



Сдвиг	MLP	TinyCNN	SimpleCNN
0 пикс.	89,0	90,6	92,5
2 пикс.	66,8	85,0	84,4
4 пикс.	41,0	84,4	54,0
6 пикс.	29,5	71,8	36,1

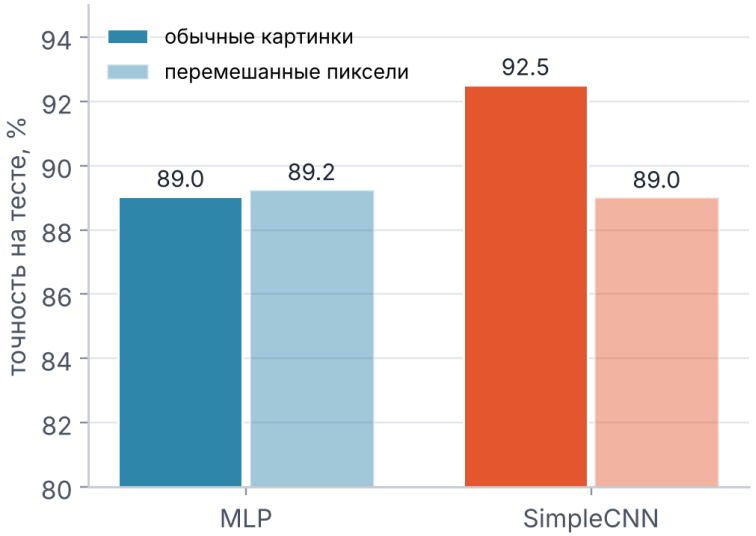
- Модели обучены без сдвигов; при проверке картинку сдвигают на d пикселей вверх, вниз, влево и вправо, освободившееся место заполняется фоном.
- **SimpleCNN** лучше MLP, но после 2–3 пикселей тоже падает: Flatten + Linear снова привязывают признаки к позициям 7×7.
- **TinyCNN** усредняет каждую карту по всей площади (GAP) и почти не теряет точность до 4 пикселей.

Свёртки эквивариантны к сдвигу; чтобы и ответ сети не зависел от положения, нужно глобальное усреднение, а не Flatten.

Эксперимент 2: перемешанные пиксели







- Все картинки — и обучающие, и тестовые — проходят через одну и ту же случайную перестановку 784 пикселей.
- Информация не теряется: перестановку можно обратить. Но соседние пиксели перестают быть соседями.

MLP

89,04 % → 89,25 %

порядок входов ему безразличен

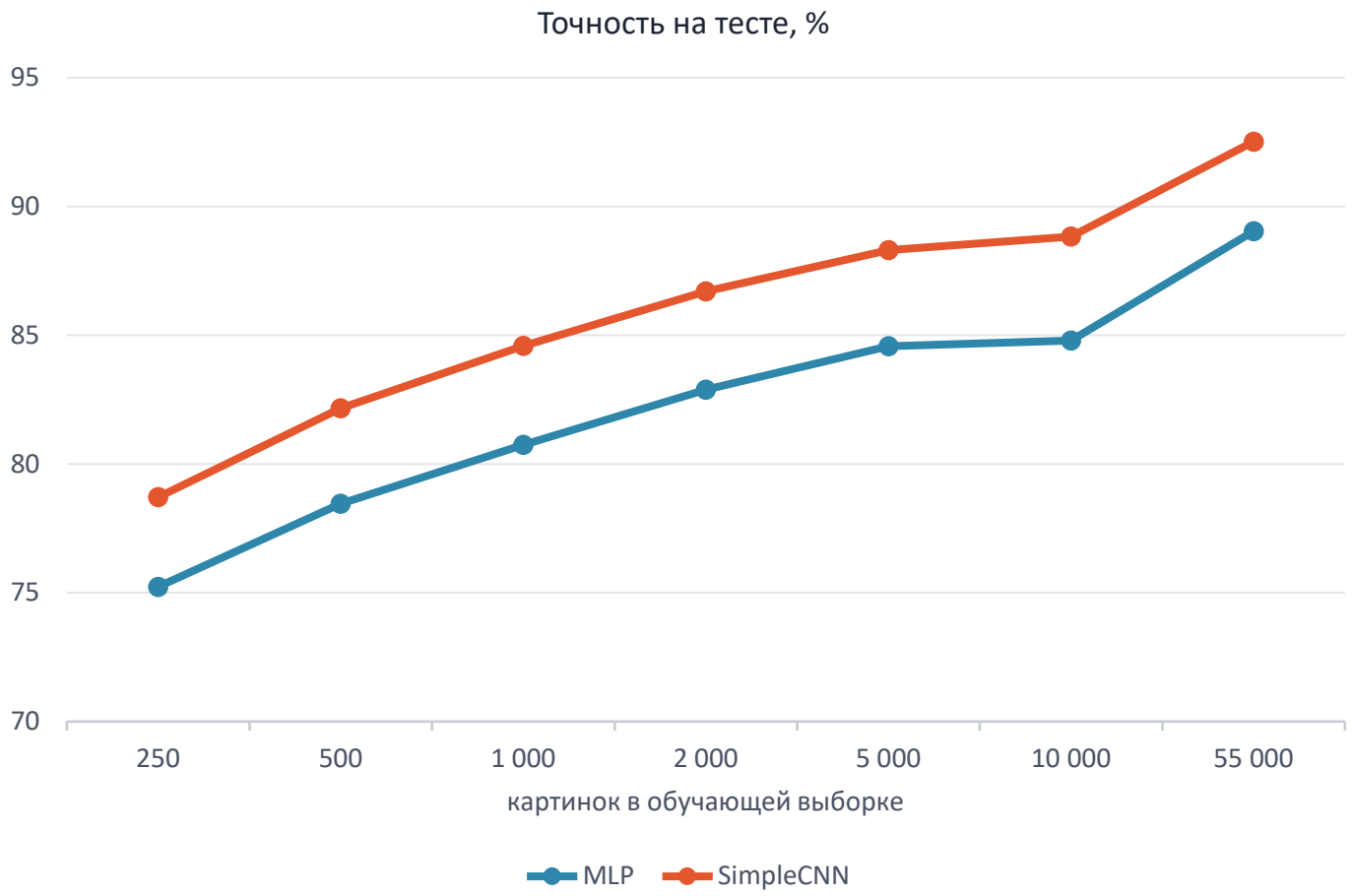
SimpleCNN

92,52 % → 89,01 %

преимущество над MLP исчезает: свёртке не на что опереться

CNN выигрывает потому, что использует структуру изображения; без неё её преимущество исчезает.

Эксперимент 3: сколько нужно данных



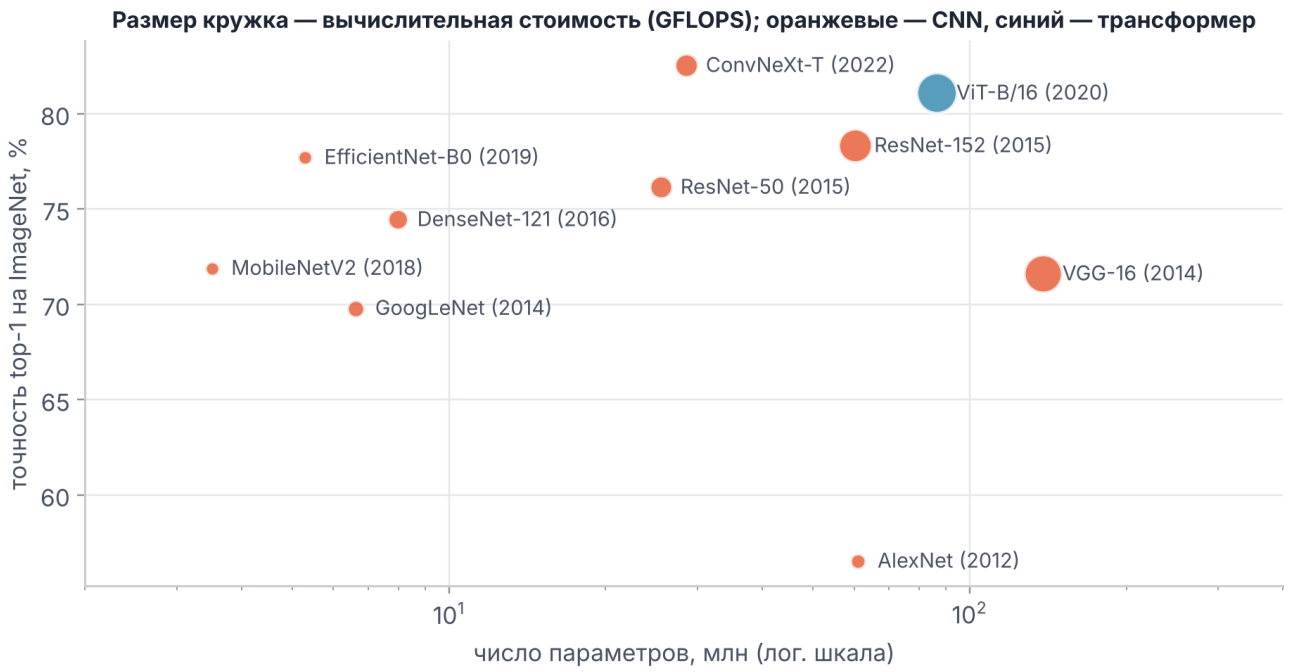
Картинок	MLP	SimpleCNN	Разница
250	75,2	78,7	+3,5
500	78,5	82,2	+3,7
1 000	80,7	84,6	+3,8
2 000	82,9	86,7	+3,8
5 000	84,6	88,3	+3,7
10 000	84,8	88,8	+4,0

Каждая точка — среднее по 3 запускам; 1000 шагов обучения при любом объёме. Последняя точка — полное обучение (12 эпох).

- При любом объёме CNN точнее на 3,5–4,0 п.п.
- CNN на **1 000** картинках (84,6 %) — как MLP на **5 000** (84,6 %).

Удачные допущения архитектуры заменяют данные: CNN достигает той же точности на выборке в 5 раз меньше.

Развитие архитектур: ImageNet



Модель, год	Ключевая идея	Пар., млн	Top-1, %
AlexNet, 2012	ReLU, Dropout, обучение на GPU	61,1	56,5
VGG-16, 2014	только свёртки 3×3, глубина 16	138,4	71,6
ResNet-50, 2015	остаточные связи y = F(x) + x	25,6	76,1
MobileNetV2, 2018	depthwise-свёртки для телефонов	3,5	71,9
EfficientNet-B0, 2019	согласованное масштабирование глубины/ширины/разрешения	5,3	77,7
ViT-B/16, 2020	трансформер на патчах 16×16, без свёрток	86,6	81,1
ConvNeXt-T, 2022	CNN, модернизированная по образцу ViT	28,6	82,5

ImageNet-1K: 1,28 млн картинок, 1000 классов. Числа — метаданные предобученных весов torchvision.

Тренд — не «больше параметров», а умнее устроенные блоки: остаточные связи, depthwise-свёртки, масштабирование.

Сравнение подходов и рекомендации

Подход	Сильные стороны	Ограничения	Когда выбирать
Логистическая регрессия, случайный лес на пикселях	быстро, просто, мало настроек	не видят пространственную структуру; ниже точность	табличные данные; быстрый базовый уровень
Ручные признаки (HOG, SIFT) + классификатор	мало данных, интерпретируемо	признаки придумывает человек; потолок качества	встраиваемые системы, очень мало данных
MLP	универсален, прост в реализации	много параметров, не устойчив к сдвигу	векторные признаки, а не изображения
CNN	локальность, общие веса, устойчивость к сдвигу, мало параметров	нужно обучение на GPU для больших задач; слабо учитывают дальние связи	**изображения, видео, спектрограммы, сигналы**
Vision Transformer (ViT)	учитывает связи между любыми частями картинки	нужно очень много данных или предобучение	большие наборы данных, предобученные модели
Предобученная CNN + дообучение	хорошо даже при сотнях картинок	нужна подходящая исходная задача	практические задачи с небольшим набором данных

■ Для изображений по умолчанию — CNN; при малом наборе данных — предобученная CNN с дообучением (transfer learning).

Заключение

1. Свёрточный слой применяет одно маленькое обучаемое ядро во всех позициях: весов в тысячи раз меньше, чем у полносвязного слоя, а признак находится в любом месте картинки.

2. Сеть складывается из простых операций — свёртка, ReLU, пулинг, — и всё обучается обычным обратным распространением; градиент по ядру — тоже свёртка (проверено вручную и в PyTorch).

3. SimpleCNN на PyTorch даёт 92,52 % на Fashion-MNIST и сама выучивает иерархию признаков: от краёв к частям вещей.

4. CNN точнее логистической регрессии, случайного леса, HOG и MLP; компактная TinyCNN (94 410 параметров) точнее MLP в 14 раз большего размера.

5. Преимущество — в индуктивном смещении: CNN (особенно с глобальным усреднением) устойчивее к сдвигу и лучше работает при малых данных, но теряет преимущество, если разрушить структуру картинки.

Спасибо за внимание! Вопросы?

Источники

- LeCun Y., Bottou L., Bengio Y., Haffner P. Gradient-based learning applied to document recognition // Proc. IEEE. 1998. Vol. 86, № 11. P. 2278–2324.
- Krizhevsky A., Sutskever I., Hinton G. ImageNet classification with deep convolutional neural networks // NeurIPS. 2012.
- He K., Zhang X., Ren S., Sun J. Deep residual learning for image recognition // CVPR. 2016.
- Dosovitskiy A. et al. An image is worth 16×16 words: Transformers for image recognition at scale // ICLR. 2021.
- Xiao H., Rasul K., Vollgraf R. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. arXiv:1708.07747, 2017.
- Goodfellow I., Bengio Y., Courville A. Deep Learning. MIT Press, 2016. Гл. 9 «Свёрточные сети».
- Dumoulin V., Visin F. A guide to convolution arithmetic for deep learning. arXiv:1603.07285, 2016.
- Zeiler M., Fergus R. Visualizing and understanding convolutional networks // ECCV. 2014 (метод перекрытия).
- Документация PyTorch (torch.nn.Conv2d, torchvision.models) — pytorch.org/docs.

Код, рисунки и числа: каталог src/ (python run_all.py).